PL/SQL

Introduction to PL/SQL Programming

PL/SQL (Procedural Language/Structured Query Language) is an extension of SQL developed by Oracle.

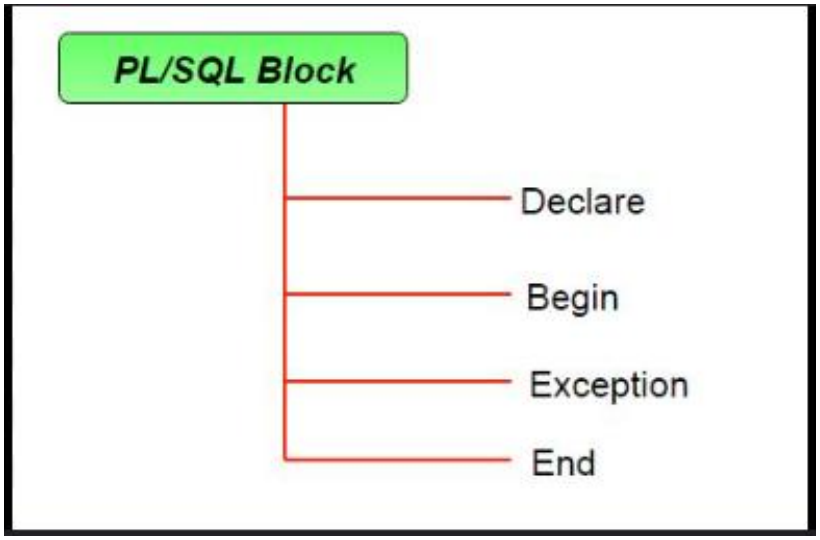
It combines the **data manipulation power of SQL** with the **procedural features** of programming languages.

It allows you to write programs that include **logic, loops, conditions, variables, functions, and procedures**, making database operations more powerful and efficient.

Key Features of PL/SQL:

- **Block-structured language** – Code is written in blocks (BEGIN…END).
- **Supports variables and constants**
- **Allows conditional statements (IF, ELSE) and loops**
- **Can handle errors using exception handling**
- **Offers better performance by reducing communication between application and database**
- **Supports stored procedures, functions, triggers, and packages**

Structure Of PL/SQL block



Simple example of PL/SQL:

```
SQL> BEGIN
  2    DBMS_OUTPUT.PUT_LINE('Welcome to PL/SQL Programming');
  3  END;
  4  /

PL/SQL procedure successfully completed.
```

Why PL/SQL is Used?

- To write complex business logic inside the database
- To improve performance by reducing repeated SQL queries
- To create reusable programs
- To ensure data security and integrity

PL/SQL data types

1. Scalar Data Types

These store single values like numbers, characters, or dates.

Data Type	Description	Example
NUMBER(p,s)	Numeric values (precision & scale)	salary NUMBER(8,2)
INTEGER	Whole numbers	count INTEGER
CHAR(size)	Fixed-length character	code CHAR(5)
VARCHAR2(size)	Variable-length character	name VARCHAR2(30)
DATE	Stores date & time	dob DATE
BOOLEAN	TRUE/FALSE	flag BOOLEAN

2. Composite Data Types

These can store multiple values (collections or records).

Type	Description	Example
RECORD	Group of related data (like a table row)	TYPE emp_rec IS RECORD(...)
TABLE (Index-by)	One-dimensional array	TYPE num_table IS TABLE OF NUMBER INDEX BY PLS_INTEGER
VARRAY	Fixed-size array	TYPE marks IS VARRAY(5) OF NUMBER

3. Reference Data Types

Used to refer to other data or objects.

Type	Description
CURSOR	Pointer to result set of a query
REF CURSOR	Dynamic cursor reference
OBJECT TYPES	User-defined objects

4. LOB Data Types (Large Objects)

Used to store large data such as files, images, documents.

Type	Description
BLOB	Binary large objects
CLOB	Character large objects
NCLOB	National character large objects
BFILE	Binary file stored externally

PL/SQL Variables and Constants

What is a Variable?

A **variable** is a named memory location used to store data temporarily that can change during program execution.

Syntax:

variable_name data_type [:= initial_value];

What is a Constant?

A **constant** is similar to a variable but **its value cannot be changed once assigned**. It must be initialized at the time of declaration using the keyword CONSTANT.

Syntax:

constant_name CONSTANT data_type := value;

```
SQL> DECLARE
  2     pi CONSTANT NUMBER := 3.14159;
  3     radius NUMBER := 5;
  4     area NUMBER;
  5  BEGIN
  6     area := pi * radius * radius;
  7     DBMS_OUTPUT.PUT_LINE('Area of Circle: ' || area);
  8  END;
  9  /
```

PL/SQL Control Structures

PL/SQL programs are built from three kinds of control flow: **sequential control**, **conditional control**, and **iterative control**. Below I'll explain each with syntax, examples, tips, and common exam-friendly notes.

There are three types of control structures:

1. Sequential control
2. Conditional statements(Decision making)
3. Iterative control

1. Sequential control (straight-line execution)

Definition:
The default mode — statements execute one after another in the order they appear, inside a block. A PL/SQL block itself is a compound (sequential) statement.

Syntax:

```
DECLARE
 -- declarations
BEGIN
 stmt1;
 stmt2;
 stmt3; -- executed in order
EXCEPTION
 -- handlers
END;
```

Example of sequential control:

```
SQL> DECLARE
  2    num1 NUMBER := 10;
  3    num2 NUMBER := 20;
  4    total NUMBER;
  5  BEGIN
  6    total := num1 + num2;
  7    DBMS_OUTPUT.PUT_LINE('Sum is: ' || total);
  8    DBMS_OUTPUT.PUT_LINE('Program End');
  9  END;
 10  /
Sum is: 30
Program End

PL/SQL procedure successfully completed.
```

2. Conditional Control (Decision Making)

Used to **execute statements based on conditions**.

Types:

- IF...THEN
- IF...THEN...ELSE
- IF...ELSIF...ELSE
- CASE

Example of if-elsif statement:

```
SQL> DECLARE
  2     marks NUMBER := 85;
  3  BEGIN
  4     IF marks >= 90 THEN
  5        DBMS_OUTPUT.PUT_LINE('Grade A');
  6     ELSIF marks >= 75 THEN
  7        DBMS_OUTPUT.PUT_LINE('Grade B');
  8     ELSE
  9        DBMS_OUTPUT.PUT_LINE('Grade C');
 10     END IF;
 11  END;
 12  /
Grade B

PL/SQL procedure successfully completed.
```

Example of case statement:

```
SQL> DECLARE
  2     marks NUMBER := 82;
  3     grade VARCHAR2(10);
  4  BEGIN
  5     grade := CASE
  6                 WHEN marks >= 90 THEN 'A'
  7                 WHEN marks >= 75 THEN 'B'
  8                 WHEN marks >= 60 THEN 'C'
  9                 ELSE 'Fail'
 10              END;
 11
 12     DBMS_OUTPUT.PUT_LINE('Marks: ' || marks);
 13     DBMS_OUTPUT.PUT_LINE('Grade: ' || grade);
 14  END;
 15  /
Marks: 82
Grade: B

PL/SQL procedure successfully completed.
```

3. Iterative Control (Loops)

Used when a set of statements needs to be executed multiple times.

Types:

- LOOP ... END LOOP; (Basic loop)
- WHILE LOOP
- FOR LOOP

Example code using LOOP…END LOOP

```
SQL> DECLARE
  2     x NUMBER := 1;
  3  BEGIN
  4     LOOP
  5        DBMS_OUTPUT.PUT_LINE('LOOP Value: ' || x);
  6        x := x + 1;
  7        EXIT WHEN x > 5;
  8     END LOOP;
  9  END;
 10  /
LOOP Value: 1
LOOP Value: 2
LOOP Value: 3
LOOP Value: 4
LOOP Value: 5

PL/SQL procedure successfully completed.
```

Example code using WHILE LOOP

```
SQL> DECLARE
  2     num NUMBER := 1;
  3  BEGIN
  4     WHILE num <= 5 LOOP
  5        DBMS_OUTPUT.PUT_LINE('WHILE Loop Value: ' || num);
  6        num := num + 1;
  7     END LOOP;
  8  END;
  9  /
WHILE Loop Value: 1
WHILE Loop Value: 2
WHILE Loop Value: 3
WHILE Loop Value: 4
WHILE Loop Value: 5

PL/SQL procedure successfully completed.
```

Example code using FOR LOOP

```
SQL> BEGIN
  2     FOR i IN 1..5 LOOP
  3        DBMS_OUTPUT.PUT_LINE('FOR Loop Value: ' || i);
  4     END LOOP;
  5  END;
  6  /
FOR Loop Value: 1
FOR Loop Value: 2
FOR Loop Value: 3
FOR Loop Value: 4
FOR Loop Value: 5

PL/SQL procedure successfully completed.
```

EXCEPTION HANDLING IN PL/SQL

What is Exception Handling?

Exception handling in PL/SQL is used to handle errors during program execution and prevent abnormal termination.

Exceptions are of **two types**:

- ✅ *Predefined Exceptions*
- ✅ *User-Defined Exceptions*

1. Predefined Exceptions

These are built-in exceptions provided by Oracle. They occur automatically when a specific error happens.

Some Common Predefined Exceptions:

Exception Name	When It Occurs
NO_DATA_FOUND	A SELECT statement returns no row
ZERO_DIVIDE	Division by zero
TOO_MANY_ROWS	SELECT returns more than one row
VALUE_ERROR	Data type conversion error

Example code using predefined-exception:

```
SQL> DECLARE
  2     num NUMBER := 10;
  3     result NUMBER;
  4  BEGIN
  5     result := num / 0;  -- This will cause ZERO_DIVIDE exception
  6     DBMS_OUTPUT.PUT_LINE('Result: ' || result);
  7  EXCEPTION
  8     WHEN ZERO_DIVIDE THEN
  9        DBMS_OUTPUT.PUT_LINE('Error: Division by zero is not allowed!');

 10  END;
 11  /
Error: Division by zero is not allowed!

PL/SQL procedure successfully completed.
```

2. User-Defined Exceptions

These are custom exceptions created by the programmer using the EXCEPTION keyword.

Steps:

1. Declare the exception
2. Raise it using RAISE
3. Handle it in the EXCEPTION block

```
SQL> DECLARE
  2     age NUMBER := 15;
  3     invalid_age EXCEPTION;  -- Step 1: Declare
  4  BEGIN
  5     IF age < 18 THEN
  6        RAISE invalid_age;   -- Step 2: Raise
  7     END IF;
  8
  9     DBMS_OUTPUT.PUT_LINE('Eligible for voting');
 10  EXCEPTION
 11     WHEN invalid_age THEN      -- Step 3: Handle
 12        DBMS_OUTPUT.PUT_LINE('Error: Age must be 18 or above to vote.');

 13  END;
 14  /
Error: Age must be 18 or above to vote.

PL/SQL procedure successfully completed.
```

CURSOR

What is a Cursor?

A **cursor** is a pointer to the result set of a SQL query. It helps PL/SQL retrieve and process rows one at a time.

There are two types of cursors:

1.Implicit Cursor

2.Explicit Cursor

Types of Cursors

1. Implicit Cursor

Automatically created by Oracle when an SQL statement (like INSERT, UPDATE, DELETE, or single-row SELECT) is executed.

No need to declare, open, fetch, or close. Oracle handles it automatically.

Example code of implicit cursor:

```
SQL> DECLARE
  2     emp_count NUMBER;
  3  BEGIN
  4     SELECT COUNT(*) INTO emp_count FROM employees;
  5     DBMS_OUTPUT.PUT_LINE('Total Employees: ' || emp_count);
  6  END;
  7  /
Total Employees: 3

PL/SQL procedure successfully completed.
```

In this example, emp table contains 3 rows. So it gave output as total employees=3

Here is the employee table:

```
SQL> select * from employees;

    EMP_ID EMP_NAME                         SALARY
---------- ------------------------------ ----------
       101 John                            50000
       102 Mahi                            60000
       103 Raj                             45000
```

If we add more tuples(rows), it will show output as per row count.

2. Explicit Cursor

Used when you want to process **multiple rows** returned by a SELECT query.

You must **manually control** the cursor with:

1. **Declare**
2. **Open**
3. **Fetch**
4. **Close**

Steps of Explicit cursors:

1. Declaring Cursor

```
CURSOR cursor_name IS
SELECT columns FROM table WHERE condition;
```

2. Opening Cursor

```
OPEN cursor_name;
```

3. Fetching Data

```
FETCH cursor_name INTO variable_list;
```

4.Closing cursor

```
CLOSE cursor_name;
```

Example of Explicit Cursor

```
SQL> DECLARE
  2     -- Step 1: Declare a cursor
  3     CURSOR emp_cursor IS
  4         SELECT emp_id, emp_name, salary
  5         FROM employees
  6         WHERE salary > 30000;
  7
  8     -- Step 2: Declare a record to hold the fetched row
  9     emp_record emp_cursor%ROWTYPE;
 10  BEGIN
 11     -- Step 3: Open the cursor
 12     OPEN emp_cursor;
 13
 14     -- Step 4: Loop through the cursor
 15     LOOP
 16         FETCH emp_cursor INTO emp_record;  -- Fetch row into record
 17         EXIT WHEN emp_cursor%NOTFOUND;     -- Exit when no more rows
 18
 19         -- Step 5: Process the data (here we just print)
 20         DBMS_OUTPUT.PUT_LINE('ID: ' || emp_record.emp_id ||
 21                              ', Name: ' || emp_record.emp_name ||
 22                              ', Salary: ' || emp_record.salary);
 23     END LOOP;
 24
 25     -- Step 6: Close the cursor
 26     CLOSE emp_cursor;
 27  END;
 28  /
ID: 101, Name: John, Salary: 50000
ID: 102, Name: Mahi, Salary: 60000
ID: 103, Name: Raj, Salary: 45000
ID: 102, Name: Sita, Salary: 35000
ID: 103, Name: Mahesh, Salary: 45000
ID: 105, Name: Raj, Salary: 50000

PL/SQL procedure successfully completed.
```

PROCEDURE

Definition of a Procedure

A **procedure** in PL/SQL is a **named block of PL/SQL code** that performs a specific task.

- It is similar to a **function**, but it **does not return a value directly** (it can return values using OUT parameters).
- Procedures can be **stored permanently in the database** (stored procedures) or **temporary** (anonymous/unstored procedures).
- They help to **modularize code**, **reuse logic**, and **simplify programming**.

Key Points

1. It is a **subprogram** that performs a task.
2. Can have **parameters** (IN, OUT, IN OUT).
3. Can be **stored in the database** or **anonymous**.
4. Improves **code reusability**, **security**, and **performance**.

Basic syntax of procedure:

```
CREATE OR REPLACE PROCEDURE procedure_name (
   parameter1 IN datatype,   -- optional
   parameter2 OUT datatype   -- optional
)
IS
BEGIN
   -- PL/SQL statements
   NULL;  -- placeholder
END procedure_name;
/
```

```
SQL> CREATE OR REPLACE PROCEDURE greet_user
  2  IS
  3  BEGIN
  4      DBMS_OUTPUT.PUT_LINE('Hello, Mahi!');
  5  END greet_user;
  6  /

Procedure created.

SQL> BEGIN
  2      greet_user;
  3  END;
  4  /
Hello, Mahi!

PL/SQL procedure successfully completed.
```

There are **two types** of procedure:

1.Stored procedure

2.Unstored (Anonymous procedure)

Stored Procedure

A **stored procedure** is a **PL/SQL block stored in the database** that can be executed repeatedly by applications or users.

Key Points:

- Saved permanently in the database.
- Can take **input/output parameters**.
- Can be called multiple times without rewriting the code.
- Improves **performance**, **reusability**, and **security**.
- Needs to be explicitly created using CREATE PROCEDURE.

Advantages of Stored Procedures

1. **Reusability** – Write once and use many times.
2. **Performance** – Stored in compiled form in the database, so faster than ad-hoc SQL.

3. **Modularity** – Break complex tasks into smaller procedures.
4. **Security** – You can restrict direct access to tables; users can run procedures instead.
5. **Maintainability** – Easy to update logic without changing the application code.
6. **Reduced network traffic** – Only the call is sent, not the whole SQL block.

Syntax to create stored procedure:

```
CREATE OR REPLACE PROCEDURE procedure_name (
   parameter1 IN datatype,      -- Optional input parameter
   parameter2 OUT datatype,     -- Optional output parameter
   parameter3 IN OUT datatype   -- Optional input-output parameter
)
IS
BEGIN
   -- PL/SQL statements
   NULL;  -- Placeholder if no statements
END procedure_name;
/
```

Example code of stored procedure.

In this code I have also written code to execute procedure and get output:

```
SQL> CREATE OR REPLACE PROCEDURE add_numbers(
  2      num1 IN NUMBER,
  3      num2 IN NUMBER,
  4      sum OUT NUMBER
  5  )
  6  IS
  7  BEGIN
  8      sum := num1 + num2;
  9  END add_numbers;
 10  /

Procedure created.

SQL> DECLARE
  2      result NUMBER;
  3  BEGIN
  4      add_numbers(10, 20, result);   -- Call procedure
  5      DBMS_OUTPUT.PUT_LINE('Sum: ' || result);
  6  END;
  7  /
Sum: 30

PL/SQL procedure successfully completed.
```

In stored procedure we can also execute procedure using EXEC command:

```
SQL> VARIABLE res NUMBER
SQL> EXEC add_numbers(15, 25, :res);

PL/SQL procedure successfully completed.

SQL> PRINT res

       RES
----------
        40
```

To delete a procedure, here is a syntax:

```
DROP PROCEDURE procedure_name;
```

Unstored Procedure (Anonymous Procedure)

An **unstored procedure** is a **PL/SQL block that is not stored in the database**.
It is written and executed **on-the-fly** (ad-hoc), usually inside an anonymous block.

Key Points:

- **Not saved** in the database.
- Can be executed **once** and disappears after execution.
- Good for **one-time tasks** or testing.
- Does **not need CREATE PROCEDURE**

Syntax for unstored procedure:

```
DECLARE
   -- Optional: declare variables here
   variable_name datatype := initial_value;
BEGIN
   -- PL/SQL statements go here
   NULL;  -- Placeholder if no statements
END;
/
```

Example code of unstored procedure.

```
SQL> DECLARE
  2      emp_count NUMBER;
  3  BEGIN
  4      SELECT COUNT(*) INTO emp_count FROM employees;
  5      DBMS_OUTPUT.PUT_LINE('Total Employees: ' || emp_count);
  6  END;
  7  /
Total Employees: 8

PL/SQL procedure successfully completed.
```

FUNCTION

What is a Function?

A **function** is a **named PL/SQL block** that performs a specific task **and returns a single value** using the RETURN statement.

- Functions are usually used when you need a **value to be returned**.
- Like procedures, functions can also take **parameters**.

Advantages of Functions

1. **Reusability** – Write once, call many times.
2. **Return Values** – Unlike procedures, functions always return a value.
3. **Modularity** – Break complex tasks into small functions.
4. **Improved Performance** – Stored in compiled form in the database.
5. **Can be used in SQL Statements** – Functions can be called in SELECT, WHERE, etc.

Syntax to create a function:

```
CREATE OR REPLACE FUNCTION function_name (
   param1 IN datatype,     -- optional
   param2 IN datatype      -- optional
)
RETURN return_datatype
IS
   -- Declaration section (optional)
BEGIN
   -- Executable section
   RETURN some_value;      -- Must return a value
END function_name;
/
```

```
SQL> CREATE OR REPLACE FUNCTION add_two_numbers(
  2       num1 IN NUMBER,
  3       num2 IN NUMBER
  4  ) RETURN NUMBER
  5  IS
  6  BEGIN
  7       RETURN num1 + num2;
  8  END add_two_numbers;
  9  /

Function created.

SQL> DECLARE
  2      result NUMBER;
  3  BEGIN
  4      result := add_two_numbers(10, 20);
  5      DBMS_OUTPUT.PUT_LINE('Sum: ' || result);
  6  END;
  7  /
Sum: 30

PL/SQL procedure successfully completed.
```

Syntax to delete a Function:

```
DROP FUNCTION function_name;
```

TRIGGERS

What is a Database Trigger?

A **database trigger** is a **PL/SQL block that automatically executes** in response to certain **events on a table or view**.

- It's **automatic**, so you don't need to call it explicitly.
- Triggers are used to **enforce business rules, maintain audit trails, or validate data**.

Use of Database Triggers

1. **Data Validation** – Prevent invalid data from being inserted/updated.

2. **Auditing / Logging** – Track changes in tables automatically.
3. **Automatic Calculation** – Update derived values automatically.
4. **Enforcing Business Rules** – Apply rules at the database level.
5. **Preventing Actions** – Stop certain operations under specific conditions.

Types of triggers:

Trigger Type	When It Fires	Example Use Case	Syntax / Mini Example
BEFORE INSERT	Before inserting a row	Validate data before insert	CREATE TRIGGER trg BEFORE INSERT ON employees FOR EACH ROW BEGIN NULL; END; /
AFTER INSERT	After inserting a row	Log new employee insertion	CREATE TRIGGER trg AFTER INSERT ON employees FOR EACH ROW BEGIN DBMS_OUTPUT.PUT_LINE(:NEW.emp_name); END; /
BEFORE UPDATE	Before updating a row	Check or modify data before update	CREATE TRIGGER trg BEFORE UPDATE ON employees FOR EACH ROW BEGIN :NEW.salary := :NEW.salary*1.1; END; /
AFTER UPDATE	After updating a row	Audit old and new values	`CREATE TRIGGER trg AFTER UPDATE ON employees FOR EACH ROW BEGIN DBMS_OUTPUT.PUT_LINE(:OLD.salary
BEFORE DELETE	Before deleting a row	Prevent deletion under certain conditions	CREATE TRIGGER trg BEFORE DELETE ON employees FOR EACH ROW BEGIN IF :OLD.salary>50000 THEN RAISE_APPLICATION_ERROR(-20001,'Cannot delete'); END IF; END; /
AFTER DELETE	After deleting a row	Log deleted data	`CREATE TRIGGER trg AFTER DELETE ON employees FOR EACH ROW BEGIN DBMS_OUTPUT.PUT_LINE(:OLD.emp_name
INSTEAD OF	On views instead of DML	Update view data	CREATE TRIGGER trg INSTEAD OF INSERT ON emp_view FOR EACH ROW BEGIN INSERT INTO employees

			VALUES(:NEW.emp_id,:NEW.emp_name,:NEW.salary); END; /
COMPOUND TRIGGER	Multiple timing points in one trigger	Perform actions **before & after** DML in one block	CREATE OR REPLACE TRIGGER trg_compound FOR INSERT OR UPDATE ON employees COMPOUND TRIGGER ... END; /

These are classified in following types:

1.Based on Timing

Type	When it Executes
BEFORE	Before the DML operation (INSERT, UPDATE, DELETE)
AFTER	After the DML operation
INSTEAD OF	Instead of the DML operation (used for views)

2. Based on Event

Type	Description
INSERT Trigger	Fires when a new row is inserted
UPDATE Trigger	Fires when an existing row is updated
DELETE Trigger	Fires when a row is deleted
COMPOUND TRIGGER	Allows multiple timing points in one trigger

Syntax to create a trigger:

```
CREATE OR REPLACE TRIGGER trigger_name

BEFORE|AFTER INSERT|UPDATE|DELETE

ON table_name

FOR EACH ROW

BEGIN

   -- PL/SQL statements

END;

/
```

Example 1: BEFORE INSERT Trigger

- Automatically logs a message when a new employee is inserted.

```
SQL> CREATE OR REPLACE TRIGGER before_emp_insert
  2  BEFORE INSERT ON employees
  3  FOR EACH ROW
  4  BEGIN
  5      DBMS_OUTPUT.PUT_LINE('Inserting Employee: ' || :NEW.emp_name);
  6  END;
  7  /

Trigger created.
```

:NEW.emp_name → Refers to the value being inserted in the emp_name column.

Example 2: AFTER UPDATE Trigger

- Automatically logs old and new salary when salary changes.

```
SQL> CREATE OR REPLACE TRIGGER after_salary_update
  2  AFTER UPDATE OF salary ON employees
  3  FOR EACH ROW
  4  BEGIN
  5      DBMS_OUTPUT.PUT_LINE('Salary changed from ' || :OLD.salary || ' to ' || :NEW.salary);
  6  END;
  7  /

Trigger created.
```

- **:OLD.salary** → Previous value of salary.
- **:NEW.salary** → New updated value.

Syntax to delete a Trigger:

```
DROP TRIGGER trigger_name;
```

------THE END-----